Can Preprocessing Close the Gap? A Study of Naive Bayes vs. RoBERTa for Tweet Sentiment Classification

A0251997L, A02345678Y, A0252829W, A0252459X
Group 28
Mentored by Esther Gan
{e0968397,e0958047,e0957771,e0957585,e0958417}@u.nus.edu

Abstract

This document is a template for the NUS CS4248 Natural Language Processing group project final report, which is limited to eight A4-sized pages for the main report body. This document contains the best practices for the group project final report, a marking rubric (§7) and formatting instructions for the report (§9). It also includes a list of frequently asked questions about CS4248 projects, included as Appendix B. The document itself conforms to its own specifications, and is therefore an example of what your report should look like.

You can (and should) access the live version of this file through the URL below: https://soc-n.us/cs4248-t2520-final-report-template. This version you're reading is version 250219 (19 Feb 2025).

1 Introduction

Sentiment analysis is a fundamental task in natural language processing (NLP), with applications ranging from market analysis to public opinion monitoring. Social media platforms such as Twitter present a particularly interesting setting for sentiment classification due to the informal, noisy, and highly contextual nature of the language used. Tweets often contain misspellings, abbreviations, emojis, and linguistic constructs such as negation that can significantly affect sentiment interpretation. These characteristics make Twitter sentiment classification a challenging benchmark for machine learning models.

Traditional machine learning approaches such as Naive Bayes have long been used for text classification tasks due to their simplicity, efficiency, and relatively strong performance on bag-of-words representations. However, Naive Bayes relies on strong independence assumptions such as the Bag of Words (BoW) assumption, which limits its ability to capture linguistic phenomena such as negation, contextual dependencies, and word interactions. In contrast, modern deep learning models such as RoBERTa have become state-of-the-art(SOTA) for sentiment analysis by leveraging contextual embeddings and large-scale pretraining.

Despite the superior performance of transformer-based models, classical algorithms like Naive Bayes remain attractive due to their interpretability, computational efficiency, and ease of deployment. An important question is therefore how far such classical models can be improved through careful preprocessing and feature engineering. In particular, preprocessing techniques such as negation handling, lemmatization, contraction expansion, filtering of noisy tokens, and custom handling of internet-specific language features may help mitigate some of the limitations of the BoW assumption.

In this project, we investigate the strengths and limitations of different approaches to sentiment classification on Twitter data. We first implement a baseline end-to-end Naive Bayes pipeline and evaluate its performance. We then develop a specialised variant of Naive Bayes that incorporates extensive preprocessing designed to address common weaknesses of BoW models, particularly with respect to negation and linguistic variation. In order to uncover the remaining weaknesses, we train RoBERTa, a SOTA model, on the same dataset, and inspect the instances where RoBERTa predicts the right label but Naive Bayes fails. From this, we come up with further preprocessing features to improve the performance of Naive Bayes.

Through this comparison, we aim to identify the extent to which preprocessing can compensate for the structural limitations of Naive Bayes, and to highlight the types of linguistic patterns that remain difficult for such models to capture.

2 Related Work / Background

2.1 State of the Art Models

In the industry, there have been many models used to do sentiment analysis on tweets. Most of the top performing models use LSTM-type models, according to nlpprogress.com[1].

As shown in this website, the best sentiment analysis model (up to 2019)[2] had an accuracy of 96%, using a self-attention setup. The only preprocessing done was just encoding relative positioning in the vector representation.

However, as transformers are more computationally expensive and harder to interpret, we wanted to see if the standard Naive Bayes with preprocessing was enough to rival the performance of the best models.

2.2 Evaluating the Effectiveness of Text Pre-Processing in Sentiment Analysis (2022) by Palonimo et al.

However, significant research has also been done in the industry on improving preprocessing methods.

In this paper[3] by Palonimo et al, where they showcased a Naive Bayes Classifer used on tweets, and tested various preprocessing pipelines to test the effectiveness of diifferent preprocessing pipelines. They have also provided a table of their accuracy scores they obtained with different preprocessing pipelines, as well as with no preprocessing, which allows for a good benchmark for our models.

In both cases, it was shown that Naive Bayes managed to obtain a very high accuracy, with as high as 91% on gold standard datasets, and 86% on the whole dataset overall. This highlights its potential to be a decent classifier despite its simplicity. 91% is also very close to the self-attention model's accuracy, at 96%, suggesting that it might be able to come close to the accuracy of more sophisticated models.

However, the tweets they used were from the political domain, about the Grand Old Party (GOP) held among the candidates of the Republican Party for the 2016 US Presidential Election. Naturally, this makes the tweets much more polarizing, making it easier for those models to achieve better scores.

With our much more neutral and nuanced dataset from Kaggle, our goal is to see if Naive Bayes could still hold up against a dataset with not so polarizing comments.

2.3 Implementation of Naive Bayes Algorithm on Sentiment Analysis Application (2021) by Martiti et al.

In this paper[4], there is yet another sentiment analysis classifier made with simple Naive Bayes, but this time done on Grab Twitter comments. This is done with TF-IDF vectors instead of Count vectors. As shown, they managed to achieve an impressive 99.623% accuracy on their Naive Bayes classifier, which highlights the massive potential in just using Naive Bayes.

However, as the dataset is different (Tweet comments about Grab instead of general tweets), which are usually more polarizing by nature, it might not reflect the actual potential of just using Naive Bayes on more neutral tweets. Furthermore, as our tweets cover a much wider range of context, rather than just one (Grab), it will likely be more difficult to just use Naive Bayes on the tweet dataset. As such, we want to try to push Naive Bayes as far as we can with our dataset.

2.4 Summary Comments on Industry Projects

From these industry projects, we thus have hope that we can simply push Naive Bayes to its limit in doing sentiment analysis on tweets, and thus will be exploring ways to improve the accuracy of Naive Bayes with preprocessing methods.

3 Corpus Analysis & Method

The dataset consists of tweet texts labelled with one of three sentiment classes: negative, neutral, or positive. The raw training set contains 27,481 rows and the raw test set contains 4,815 rows. After removing rows with missing values in critical fields such as text and sentiment, 1 row was dropped from the training set and 1,281 rows were dropped from the test set, leaving 27,480 training rows and 3,534 test rows. Within the training set, 11117 rows have neutral sentiment, 8582 have positive sentiment and 7781 have negative sentiment. As the distribution of sentiment seems to be rather balanced with no significant minority, we chose not to

[1]The website used: https://nlpprogress.com/english/sentiment_analysis.html.

[2]The model used: https://arxiv.org/pdf/1906.08237.

[3]The paper by Palonimo et al.: https://www.mdpi.com/2076-3417/12/17/8765.

[4]The paper by Martiti et al.: https://www.atlantis-press.com/article/125963764.pdf.

apply class-balancing techniques as oversampling and undersampling incur their own tradeoffs.

In addition to tweet text, the dataset includes metadata such as country, age group, time of tweet, population, land area, and density. However, our exploratory analysis suggested that these metadata features are weak predictors of sentiment. Sentiment distributions remained broadly balanced across these categories so we chose to focus primarily on the tweet text itself rather than these auxiliary variables.

Note that the value present in "selected_text" is a subset of the value present in "text". While the process of deriving "selected_text" is not explicitly documented in the dataset, the assumption is that it represents the most relevant part of the tweet that led to the classification result. Regardless, this column is not present in the test set and hence excluded from training.

As our goal is to analyse how model architectures affect performance, we apply the same preprocessing pipeline to all three models.

1. **Removal of rows with critical missing values.** Rows with missing values in the "text" and "sentiment" columns are removed. It was manually observed that all the testing rows that are dropped are merely empty rows caused by formatting issues.

2. **Lowercasing.** All tokens are converted to lowercase to reduce vocabulary sparsity by allowing a single set of weights per word to be learned (e.g., "Good" and "good" are mapped to the same feature). While this discards potential signals from ALL CAPS that signals against a neutral classification, our experiments indicate that the signal was largely inconsistent so the generalization gain outweighted the potential information loss.

3. **Translation of emoticons.** Emoticons are translated to their corresponding text representation to reduce vocabulary sparsity.

4. **Removal of numbers.** Numeric tokens are removed because they rarely carry stable sentiment signals in the corpus and can introduce noise into bag-of-words features (e.g., for the sentence "I earned 100 dollars after a day's work", it is uncertain whether 100 is a large enough number for this sentence to be positive or negative).

5. **Negation tagging.** We explicitly model local negation scope by prefixing the next N tokens with `NOT_` after a negation trigger (e.g., *not good at all* $\rightarrow$ *NOT_good NOT_at NOT_all* for N=3). This helps the model distinguish polarity reversals such as "good" versus "not good".

6. **Removal of URLs.** URLs are removed as they are not informative for sentiment classification and can introduce noise into the feature space.

7. **Removal of expressive lengthening.** Expressive lengthening such as "soooo" could be replaced with "soo" to reduce the vocabulary space. The logic involves replacing repeated characters of more than length 2 with only 2 characters. Our experiments indicate that performance gains caused by a smaller vocabulary space did not outweigh the loss of sentiment information caused by the removal of expressive lengthening, indicating that the expressive lengthenings are meaningful indicators of sentiment.

The following pre-processing steps are removed from the preprocessing pipelines as they did not improve the performance of the models:

1. **Removal of stopwords.** The performance of the model worsened after experimenting with both the nltk stopword list and a custom-created list that preserves words which signal sentiment. This may be because stopwords can carry implicit structural and contextual cues that the model relies on, and their removal disrupts the natural flow of text, leading to a loss of information that is difficult to recover.

2. **Lemmatization.** Lemmatization was empirically observed to have worsened performance despite the potential benefits of reducing the vocabulary space. A likely reason is that morphological variations (e.g., verb tenses, plural forms) can themselves carry sentiment-relevant information — for instance, *loved* versus *love* — which is erased when words are reduced to their base form.

4 Experiments

Exploratory data analysis showed that the sentiment labels were relatively balanced across the

three classes, so we did not apply class-balancing techniques such as oversampling or undersampling. We also examined several linguistic properties of the tweets. The corpus contained substantial numbers of URLs, hashtags, emoticons, and elongated expressions, confirming that the dataset reflects the noisy and informal nature of social media text.

While the dataset contains additional metadata such as country, age group, and time of tweet, our modelling pipeline focuses exclusively on textual features. Therefore, we restrict our models to operate on tweet text.

4.1 Naive Bayes method

For the classical baseline, we trained a Multinomial Naive Bayes classifier. Text was vectorized using a count-based representation. We performed hyperparameter grid search using sklearn's RandomizedSearchCV and GridSearchCV functions over the vectorizer and model settings, including the n-gram range, minimum document frequency, vocabulary size, and smoothing parameter alpha. Starting from a wide range for each feature, We did randomized grid search a few times, and slowly zoomed in to test more precise values.

The best-performing configuration and the test ranges used are presented in Table 1.

The optimal n-gram range indicates that unigram and bigram features were sufficient, while more complex n-gram ranges did not yield better cross-validation performance in the saved notebook.

Hyperparameter	Value	Test Range
Max Features	6500	1000-30000
N-gram Range	(1,2)	(x,y); $x, y \in$ [1,5]
Minimum df	1	1-5
Alpha	1.0	0.1-1.0
Vectorizer	Count	Count, TF-IDF

Table 1: Best-performing Naive Bayes hyperparameters

Unlike standard multiclass classification, our notebook maps sentiment labels to ordinal scores -1, 0, 1 and also experiments with threshold calibration for converting prediction scores back to discrete classes. This was done to improve macro-F1, especially for the neutral class, which often lies between clearly positive and clearly negative examples.

4.2 Naive Bayes results

Using default sentiment thresholds of -0.33 and 0.33, the Naive Bayes model achieved an overall accuracy of 0.69 and a macro-F1 score of 0.69 on the test set. Class-wise results showed that positive tweets were the easiest to classify, while neutral tweets were more difficult.

We additionally performed threshold calibration on the predicted sentiment scores to improve macro-F1. By tuning decision boundaries for negative and positive classes, we observed a small improvement in macro-F1, suggesting that the neutral class benefits from more flexible decision thresholds. This calibration slightly improved the balance across classes, particularly by increasing neutral-class recall, although overall accuracy remained around 0.69.

Hyperparameter tuning further improved the accuracy from about 0.69 to 0.71, suggesting that the model is already highly optimised, and that the limitations come from the model architecture or the dataset.

These results suggest that even a relatively simple bag-of-words model can perform competitively when supported by suitable preprocessing and decision-threshold tuning.

4.3 RoBERTa method

To compare against a pretrained transformer, we fine-tuned DistilRoBERTa on the sentiment dataset. Instead of directly treating the task as a 3-way classification problem, the notebook formulates it as regression over sentiment scores, where negative, neutral, and positive are mapped to -1, 0, and 1 respectively. The regression outputs are then converted back into class labels using thresholding.

The model used a maximum sequence length of 64, batch size of 32, learning rate of 2e-5, weight decay of 0.01, and one training epoch. This design keeps training lightweight while still leveraging contextual representations learned from large-scale pretraining.

Compared with Naive Bayes, RoBERTa is expected to better capture contextual sentiment cues, especially cases involving negation, composition, and longer dependencies that are difficult to represent with bag-of-words features alone.

4.4 LSTM method

We also implemented an LSTM-based classifier as an intermediate neural baseline between Naive

Bayes and RoBERTa. The LSTM pipeline uses a custom vocabulary with minimum frequency filtering, padded mini-batches, trainable embeddings, and a bidirectional LSTM encoder followed by dropout and a linear classification layer.

The preprocessing for the LSTM includes lowercasing, lemmatization, URL removal, translation of hashtags and emoticons, and retention of stopwords. This setup is intended to preserve sentiment-bearing function words while reducing noise from informal tweet formatting.

Hyperparameter tuning was performed over embedding dimension, hidden dimension, dropout rate, batch size, and bidirectionality. The best hyperparameter setting recorded in the notebook was

Hyperparameter	Value
Embedding dimension	100
Hidden dimension	256
Dropout	0.4
Batch size	32
Number of layers	1
Bidirectional	True

Table 2: Best-performing LSTM hyperparameters

5 Discussion

Our experiments suggest that preprocessing matters substantially for tweet sentiment classification, especially for simpler models. Handling negation, preserving sentiment-bearing tokens, and calibrating thresholds allowed Naive Bayes to reach a reasonably strong macro-F1 despite its simplicity. This shows that careful text normalization and representation can partially compensate for the limitations of bag-of-words models.

At the same time, the architecture choices in the notebook reflect the expectation that neural models should perform better when contextual understanding is required. RoBERTa, in particular, should be more robust to contextual polarity shifts and compositional meaning. LSTM provides a useful middle ground, offering sequential modelling without the full computational cost of a transformer.

One limitation of the present notebook is that the saved outputs are incomplete for the neural models. This makes it difficult to perform a full empirical comparison directly from the notebook snapshot alone. In the final report, the analysis should therefore distinguish clearly between implemented methods and fully verified numerical results.

6 Conclusion

In this project, we investigated tweet sentiment classification using Naive Bayes, RoBERTa, and LSTM models. Exploratory analysis showed that textual content was far more informative than the available metadata, leading us to focus on tweet-level preprocessing and modelling. Among the saved results, Naive Bayes achieved about 0.69 accuracy and 0.70 macro-F1 after threshold calibration, demonstrating that a simple model can remain competitive with strong preprocessing. Future work should complete the evaluation of the neural models and compare their gains against their additional computational cost.

7 Report Best Practices

Your group report is **strictly** limited to eight pages[5] and summarises all of your group's understanding of your problem, models, experimental results and insights. Typically, such (empirical/experimental) scientific reports for natural language processing follows a five- to six-section format (suggested length in parentheses, for a **eight**-page limit; do not feel compelled to match these suggested lengths), as follows:

1. **Introduction** (1/2–1 page): Motivate your work, state the problem statement clearly (inputs and outputs), and summarise your key contributions. Optional to include are a concluding textual navigation paragraph, and/or running (microanalysis) example.

2. **Related Work / Background** (1/2–1 1/2 pages): Implementation and experimentation that your group did in the project should be based on others' experiences. Relate these relevant works to show that you are aware of best practices, and how your work builds upon them. Your report can call attention to gaps in the related work to motivate your work to as innovative and filling in knowledge that is lacking. This section typically features many citations to prior work and footnote references[6] to online datasets or software.

3. **Corpus Analysis & Method** (1–2 pages): Describe the different key useful approaches that

[5] For the main body of the report: title, abstract and main sections. Backmatter does not count towards this limit.

[6] Such as this one: http://nlpprogress.com/.

yielded interesting findings here. You need not include all of your group's work if certain branches did not prove useful; those you can mention in an appendix. Describe the preprocessing, data collection, and the main methods used. You may also refer to your group's software repository in a footnote, if you host it online[7].

4. **Experiments** (1–2 pages): This section gives the main experimental settings, such as the corpus used, (macroscopic) evaluations metrics and baselines first; then proceeds to show the key performance evaluation experimental results, usually through figures, tables or charts. Interpret the data in these artefacts as prose explanations in the body text.

5. **Discussion** (1–3 pages): Enumerate 2–3 specific research questions and your group's answer that give more depth and analysis to the main results. These can describe performance aspects to sub-populations of input or intended users; time, memory and compute costs and scaling; micro-analysis of specific input instances. At least one question should be related to the natural language aspect (in contrast to general machine learning) of your project and corpora.

6. **Conclusion**: (1/4–1/2 page): This section is often abused as another chance to repeat the abstract or the introduction. Use this section instead to help summarize and lend insight to the reader, in light that they now have read the contents of the other sections. Limitations of your project, future directions also feature in this final section.

Aside from these sections, there will be a short 100–200 word abstract at the beginning of the containing a summary of the work accomplished. The abstract usually contains a clear task statement, highlights of experiments and key findings of the work.

Following the eight-page maximum length report body, there is unlimited space for you to include materials — an ethical statement, references and appendices (in that order).

[7]An example would be a GitHub repository such as `https://github.com/knmnyn/cs4248-2520`. If you provide one, please ensure that you have at least a minimally-documented `README.md` file and organize your repository accordingly

It is recommended that you start with this format and permute it to your liking.

8 Marking Rubric

The marking of the project report follows a similar format to the *Intermediate Update*: Presentation, Content, and Miscellaneous. We will mark out of a total maximum mark of 100. Your grade and comments on the marking will be made available by Canvas Gradebook.

Report Presentation (25%):

- Motivation:
  - Does the report clearly outline of goals and questions addressed?
  - Is the motivation for your task clear, plausible and rational?
  - Is the problem statement well-defined using appropriate NLP terminology?
  - Does the report state the importance, usefulness, benefits of the work and the results?
- Structure
  - Does the report content flow logically?
  - Is it sufficiently well-organized to omit information that should be common knowledge to your peers?
  - Do you relegate less important information to an appropriate location (backmatter, software repository, footnote)?
- Visualization
  - Does the report use appropriate figures, plots, and tables to justify preprocessing steps, design decisions, motivating discussions and explanations?
- Presentation (more important)
  - Does the prose, references, sections and visuals all complement each other in describing the logical flow?
  - Is any corpus analysis (exploratory data analysis) done purposefully, to motivate model or experimental design?
  - Are any visuals appropriately-sized, captioned and legible? Do they serve to better explain the material than an equivalently-sized block of prose text?

- Do you correctly follow the formatting instructions, length limitations and submission rules?

Do not just report numbers, but illustrate (with figures, tables), and explain them. Do not assume that your audience knows what your numbers mean.

Report Content (60%):

- Originality
  - What are the original elements done in the project? (It's not necessary that no group has done your task before, but your report needs to reflect your ability to think analytically and contribute novel analysis.)
  - Do you articulate how your work is novel in light of the prior work?
- Relevance
  - How strongly connected is the project to this course?
  - Do you use core concepts of NLP taught from class?
- Related Work
  - How strongly connected is the project to this course?
  - Do you use core concepts of NLP taught from class?
  - Do you present a study of related work to the task? (Formal academic references, useful web articles and posts material, and other related work should be considered in this aspect. Remember to cite explicitly.)
- Technical Justification (more important):
  - Is your technical approach suitable to try to solve your proposed problem?
  - Is your technical approach valid for your task and dataset?
  - Are there technical flaws in the execution of the approach?
  - Do you describe the data / corpora that you collected in an appropriate manner? (Self-annotated data may need evidence that the annotations are replicable; i.e., interannotator agreement)
  - Are evaluations performed with the appropriate metrics and correctly interpreted?
- Implementation (more important):
  - Did you implement multiple models (baseline, and best)?
  - Do you cleanly delineate what your group members coded as original work from public library or code repositories you used from others?
  - Did you implement or use the models correctly?
  - Did you tune them appropriately, where resources allowed?
- Model Evaluation (more important):
  - Do you address both macroscopic, dataset-wide level performance (e.g., F1 measures) as well as microscopic, individual instance level performance (careful error analysis with diagnosis)?
  - Do you demonstrate improvement in performance from your model to another, such as a baseline model? (A baseline model may be an implementation of a simpler model or version of your model, or referenced from other literature — make sure to give appropriate citations).

  Note that your performance need not be very high (e.g., 90%) if your data problem is hard. But you should show improvement over some baseline approach. This includes conscientious efforts to improve performance.
- Results Interpretation (10%): How well are the evaluation results described and interpreted.
  - Error analysis: Explain, with evidence, why the model may be performing poorly (or not as good as you wish).
  - Do you justify technically why your model is good or has improved? I.e., rationalize your approach's performance effectiveness.
  - Future improvements: Discuss how you may further improve your model.

  You do not have to implement or test all your ideas, if too infeasible. Though discussing

them helps to show your grading staff that you have good and valid ideas.

Miscellaneous (15%):

- Reproducibility
  - Is the technical approach described clearly and sufficiently detailed for a peer to replicate? Is the evaluation method described clearly and detailed enough for a peer to replicate?
  - Is your source code well-organized and any ancillary materials well documented?
  - Are your results easy to replicate by running documented commands or executing a notebook?
- Limitations
  - Do you state the principal limitations of your work, such as the important aspects of the problem domain, and how these factors might be mitigated?
- Backmatter
  - Do you use the backmatter and supplemental materials (website, source code repository) effectively to complement the formal report body?
  - Are your references bibliographically complete?
  - Did your group appropriately fill out the *Statement of Independent Work*?
  - Did you properly acknowledge and document how AI tools played an appropriate role in your experimentation, coding and report?

Late policy. Please refer to the CS4248 website for late policy, in Canvas » Pages » Grading. In general, our course's late policy is harsh to help our instructing staff mark in an efficient manner. To ensure your group does well, please turn in your report on time (.PDF version to the Canvas Assignment) by the deadline. If you envision that your group cannot meet the deadline and you wish to seek an extension, please do so well in advance of the deadline, and not after the deadline.

9 Report Formatting

This document is a LaTeX sourced document, a common typesetting system used in research communities, and commonly used for many conferences for natural language processing research. As such we are using this typesetting system and the formatting (style) files common to best practices for communicating NLP research and outcomes. This template is hosted on the third-party cloud-based LaTeX typesetting site, Overleaf, which you may decide to use if you'd like. Note that NUS has a sitewide license to this product so you can get professional features when using an NUS email address (e.g., ones ending in `u.nus.edu` for free.

To be clear, it is **not necessary** for your group to use LaTeX to typeset your final report. You may use other software and reproduce (most of)[8] the template following the styles noted below.

1. If you are using a system other than LaTeX (e.g., MS Word, Open Office, LibreOffice, Apple Pages), you will want to follow the guidelines in the rest of this section for the report format.
2. If you are using Overleaf/LaTeX, you should simply be able to use the logical formatting tags directly — which should have been configured properly by the inclusion of the `acl.sty` and `acl_natbib.bst` style files — and ignore the rest of this section. Please see the LaTeX source of this document for comments on other packages that may be useful.

Format your report to two columns to a page, A4 sized page only, in the manner these instructions are formatted. The exact dimensions for a page on A4 paper are:

- All (Left, right, top and bottom) margins: 2.5 cm
- Column width: 7.7 cm
- Column height: 24.7 cm
- Gap between columns: 0.6 cm

For reasons of uniformity, Adobe's **Times Roman** font should be used.

[8] In other software, it may be difficult to reproduce the margin line numbering, this is ok to omit.

Type of Text	Font Size	Style
paper title	15 pt	bold
author names	12 pt	bold
author affiliation	12 pt	
the word "Abstract"	12 pt	bold
section titles	12 pt	bold
document text	11 pt	
captions	11 pt	
abstract text	10 pt	
bibliography	10 pt	
footnotes	9 pt	

Table 3: Final Report Font guide. Captions generally should be self-sufficient to read the table or figure independently of the prose text of the report. Use bottom and top rulelines for proper formatting.

9.1 The First Page

Center the title, author's name(s) and affiliation(s) across both columns. Do not use footnotes for affiliations. Use the two-column format only when you begin the abstract.

Title: Place the title centered at the top of the first page, in a 15-point bold font. (For a complete guide to font sizes and styles, see Table 3) Long titles should be typed on two lines without a blank line intervening. Approximately, put the title at 2.5 cm from the top of the page, followed by a blank line, then each of your groupmates' NUS student IDs, Replace the `XX` and `YY` placeholders with your two-digit Group ID (e.g., "01") and project mentor's name. Do not format title and section headings in all capitals as well except for proper names (such as "BLEU") that are conventionally in all capitals. The affiliation should an electronic mail address for at least one contact student.

Start the body of the first page 7.5 cm from the top of the page.

Abstract: Type the abstract at the beginning of the first column. The width of the abstract text should be smaller than the width of the columns for the text in the body of the paper by about 0.6 cm on each side. Center the word **Abstract** in a 12 point bold font above the body of the abstract. The abstract should be a concise summary of the general thesis and conclusions of the paper. It should be no longer than 200 words. The abstract text should be in 10 point font.

Text: Begin typing the main body of the text immediately after the abstract, observing the two-column format as shown in the present document. Do not include page numbers.

Indent when starting a new paragraph. Use 11 points for text and subsection headings, 12 points for section headings and 15 points for the title.

9.2 Sections

Headings: Type and label section and subsection headings in the style shown on the present document. Use numbered sections (Arabic numerals) in order to facilitate cross references. Number subsections with the section number and the subsection number separated by a dot, in Arabic numerals. Do not number subsubsections.

9.3 Citations

Output	natbib command
(Gusfield, 1997)	`\citep`
Gusfield, 1997	`\citealp`
Gusfield (1997)	`\citet`
(1997)	`\citeyearpar`

Table 4: Citation commands supported by the `acl.sty` style file. The style is based on the natbib package and supports all natbib citation commands.

Citations are bibliographic references to scholarly works. You may include references to documentary web pages, blog posts, reports, pre-prints as well. Note that references to general webpages or software (packages) as URLs are more appropriately given as footnotes.

Table 4 shows the syntax supported by the style files. We encourage you to use the natbib styles. You can use the command `\citet` (cite in text) to get "author (year)" citations, like this citation to a paper by Gusfield (1997). You can use the command `\citep` (cite in parentheses) to get "(author, year)" citations (Gusfield, 1997). You can use the command `\citealp` (alternative cite without parentheses) to get "author, year" citations, which is useful for using citations within parentheses (e.g. Gusfield, 1997).

9.4 Backmatter

Backmatter are other materials that follow the main report body. They include the following:

References. Your group should cite all appropriate references that you need in your report. You may place an *unlimited* number of references to work that is relevant, beyond the page limit for the main report.

The LaTeX and BibTeX style files provided roughly follow the American Psychological Association format. If your own bib file is named `custom.bib`, then placing the following before any appendices in your LaTeX file will generate the references section for you:

```
\bibliographystyle{acl_natbib}
\bibliography{custom}
```

Many papers in natural language processing come from the ACL Anthology, the digital library for NLP, which Min ran for many years. You can obtain the complete ACL Anthology as a BibTeX file from https://aclweb.org/anthology/anthology.bib.gz. To include both the Anthology and your own .bib file, use the following instead of the above.

```
\bibliographystyle{acl_natbib}
\bibliography{anthology,custom}
```

Statement of Independent Work. Your group must include this section, to declare whether your group followed class policy. Refer to the example in this document's backmatter.

Ethical Statement. An optional, unnumbered section. Refer to the example in this document's backmatter.

Acknowledgements. An optional, unnumbered section. Refer to the example in this document's backmatter.

Appendices. You are also allowed *unlimited pages* for appendices, but be aware that your teaching staff is not obligated to read or acknowledge these sources.

Use `\appendix` before any appendix section to switch the section numbering over to letters. See Appendix A for an example.

References

Rie Kubota Ando and Tong Zhang. 2005. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6:1817–1853.

Galen Andrew and Jianfeng Gao. 2007. Scalable training of L1-regularized log-linear models. In *Proceedings of the 24th International Conference on Machine Learning*, pages 33–40.

Dan Gusfield. 1997. *Algorithms on Strings, Trees and Sequences*. Cambridge University Press, Cambridge, UK.

Mohammad Sadegh Rasooli and Joel R. Tetreault. 2015. Yara parser: A fast and accurate dependency parser. *Computing Research Repository*, arXiv:1503.06733. Version 2.

Acknowledgements

Place any acknowledgements here. You can thank any people you contacted or sources that you used that are not bibliographic in nature.

This document has been adapted from the ACL Rolling Review Template (ACL ARR) by Min-Yen Kan. You may find the original template, which NUS has also contributed to in the past, here: https://www.overleaf.com/latex/templates/acl-rolling-review-template/jxbhdzhmcpdm. We have omitted much of the original document to cut down on verbiage.

Our final report grading rubric is based on a merger of guidelines from component courses CS5228 Knowledge Discovery and Data Mining and CS3244 Machine Learning.

Statement of Independent Work

You **must** *include the text of the two statements below in your group's submitted work. Digitally sign your submission using your Student Numbers (starting with A...; N.B., not your NUSNET email identifier). This is a required section and is not part of the main body (doesn't count towards your page limit).*

1A. Declaration of Original Work. By entering our Student IDs below, we certify that we completed our assignment independently of all others (except where sanctioned during in-class sessions), obeying the class policy outlined in the introductory lecture. In particular, we are allowed to discuss the problems and solutions in this assignment, but have waited at least 30 minutes by doing other activities unrelated to class before attempting to complete or modify our answers as per the class policy.

To be explicitly clear, AI use is allowed and encouraged for all assignments in CS4248. We have documented our use of AI tools (if applicable) in a following table, as suggested in the NUS AI Tools policy[9]. This particular document did not use any AI Tools to proofcheck and was constructed and edited purely by manual work.

If the production of your report used AI Tools (inclusive of Generative AI), do keep detailed logs of how you used AI Tools, as your project requires the accountability of an audit trail of your interaction(s) with such tools (prompts, output).

1B. Exception to the Class Policy. We did not follow the CS4248 Class Policy in doing this assignment. This text explains why and how we believe we should be assessed for this assignment given the circumstances explained.

Signed, [Enter your Axxx Student IDs and NUSNET email addresses here]

Ethical Statement

The optional ethical statement is an unnumbered section that comes after the references. Most projects may not need to include such a statement, but we include it here, as it is important to be aware that NLP research and experimentation needs to be conducted in an ethically acceptable manner.

It describes any pertinent issues with respect to the NL technology being described in the project work. These could include dual-use, data quality discussions, compute requirements, fair pay for annotators and evaluators, among other factors.

You may read more about these issues by reading the *Guidelines for Responsible NLP Research*[10] and consulting works on the ACL Ethics Reading List[11].

[9] https://libguides.nus.edu.sg/new2nus/acadintegrity, tab "AI Tools: Guidelines on Use in Academic Work"

[10] https://aclrollingreview.org/responsibleNLPresearch/

[11] https://github.com/acl-org/ethics-reading-list

A Example Appendix

Optional appendices are the last item in the report.

If your group's report is too long, working to best structure the core of the report (instead of technical details) in the main body and relegating details for replication in appropriate appendices is key.

Since you have unlimited pages for appendices, you can afford to make any plots or result tables larger in the appendices, but do ensure that key results are in the report's main page limit rather than relegated here.

B Project Frequently Asked Questions (FAQ)

This section is sourced from the CS3244 Machine Learning module's Project FAQ.

1. *Q: Just to be sure, for our project work, can we code in any language (i.e. R) other than python?*
   A: Yes.

2. *Q: Does the project difficulty matters for the grading, e.g. taking the* `easy` *dataset versus doing something tagged as* `hard`*? Would we be graded based on the novelty or "importance" of the use case/problem our group comes up with?*
   A: We aren't looking at technical complexity when grading the project. We state the notional technical difficulty of project dataset to help your team decide which type of project to take on. More difficult datasets usually involve more specific preprocessing, data normalization and usually (much) larger compute costs in manipulating large-scale data.

   - What we are looking at is the learning that comes out of engaging in the project. We want to see you twist your mind and come up with interesting approaches to the problem of choice.
   - We also do not place heavy emphasis on metrics like accuracy, log loss, precision, recall, etc.. We'd care more about questions like "why did you use XYZ Metric over ABC Metric for this problem?". Getting a +0.5 accuracy boost doesn't matter as much as why you chose to do ABC Technique that brought about that accuracy boost in the first place.
   - We care more about how you communicate your findings to us in an interesting way like your analysis, your wins, your losses (pun not intended), etc.. That way, it shows us that you gained valuable experience from this project that you can apply to future projects.
   - You are allowed to explore models not covered in class at your own discretion. We only teach you the fundamentals in hopes of making you comfortable with the math/concepts involved. Beyond that, you can look at more complex models and techniques not taught in CS4248 for your projects. But again, complexity is not the focus, communication and understanding the 2W1H (why, what, how) are.
   - There isn't any true "novelty" in these projects *per se*. They are popular benchmarks found in the real world with increasing difficulty of use. We want you to have your own unique spin to these solutions (please do not copy-paste/plagiarise someone else's code from online) and present them in a way you and we (i.e. the teaching staff) understand.

   These projects are for you in the long run, not us. Hope this helps.

3. *Q. Have any advice for experimentation?*
   A: Sure. We recommend staging your experiments' time to execute to fit your working style. For example, we recommend having 3 granularities of time for your experiment execution: immediate (finishes within 1–3 minutes), coffee/tea break (finishes within 30-60 mins), overnight (as the name implies). Based on some initial runs, you should develop a good estimation for how long your pipeline takes for a certain data scale, and retrofit/sample data from your dataset to fit accordingly.

   Immediate experiments should just to check that your code works with a tiny toy dataset without faults and to assess whether an experimental setting can be escalated to the next granularity.

   Coffee / Tea Break experiments test those runs from the Immediate scale that reach your standard for trying on a larger dataset. These experiments can be set to run on a server with a medium-sized dataset that can complete independently while you are eating a meal, or taking a break to do other work or play. These validate your ideas on larger scale datasets without committing to training an entire dataset without knowing whether the results point appropriately in the proper direction – Min has seen many times that the "math works out (e.g., shape of matrices are fine) but which the computation is garbage (e.g., one off indexing errors) – so this scale mitigates this. These scale experiments need to be followed up with analysis to ensure that the results are as expected and appropriate.

   Overnight (or longer) experimentation runs your training or testing at scale, for production or for final presentations or reports. Try not to do this scale of experimentation without having a good reason to believe it will succeed (i.e., don't run a large-scale experiment to try something out; you should have done that at the Coffee / Tea Break scale instead).

4. *Q: Is model performance and using and getting state-of-the-art performance an important output of our project?*
   A: Generally, no. Good projects explain and teach, rather than just show good results. It is better to use simpler models where you can show that your understanding of the model, features, corpus and evaluation metrics interact to lead to the performance levels you observe. Merely swapping in a newer more advanced model and getting better results doesn't merit this understanding. Good and replicable results are nice-to-have in terms of grading criteria, but not must-haves.

5. *Q: Just want to check if our group's understanding of an ablation study is correct. If we have around 20 features that we engineered and a baseline model of previously proposed features, how do we efficiently conduct an ablation study? We are thinking of grouping the 20 features we have into a few groups to turn them on/off for the study. Is this the correct way? Should our angle start out from all features included to removing parts of it, or start from the baseline model and add features, since it's an "ablation" study.*

A: Yes, that would be appropriate. You can turn off a feature group, or some pre-/post- processing and see what the effects are. Before you do that, your team should hypothesize what you think would happen. This can help you hone your sense of understanding. A scientist does experimentation with a hypothesis in mind. Generally you have a final model and you turn off certain features to study their (negative) impact on your final model. This allows you to argue for the necessity of all feature groups in your model.

6. *Q: I have some questions regarding the ablation studies we need to do for the project. Based on my understanding, the aim of ablation studies is to understand how our existing system (i.e. the feature engineering techniques+the models) work. And I remember you saying that it is about having hypothesis and finding ways to test them. So my first question is, do we have to have some form of "removal" to conduct ablation analysis? Or is it ok to analyse without "removal"?*
   A: Yes that's correct. No you need not (always) do a removal (ablation) for analysis. It is just a common form. Both additive and ablative (removal) studies of feature classes are common. Ablation studies often form the basis for arguing that the model cannot have any of its elements removed without damaging a performance metric.

7. *Q: If the aim is to understand how our existing system works, how do we control the factors of the experiment? For example, if feature engineering technique A works well with a model, and we wanna know what is in A that makes this succeed, so we modify technique A into its variant C (with one feature removed), and then here's the question: do we feed the input processed using C into the model trained using A and see the change of results, or do we train a new model of the same structure on this input processed using C and then compare the results with inputs processed by A feeding into model trained using A?*
   A: Yes, sometimes we refer to C as "C: Model A–<some feature>". The second method, train a new model. We compare the performance macroscopically (whole dataset, e.g., accuracy, F_1) against the system trained with the output from A.

8. *Q: It was mentioned at the start of the sem that there was no real novelty in our projects, but there was novelty of project in the project presentation rubrics. Could you please clarify what novelty in the marking rubric truly stands for?.*
   A: Indeed with many of our curated datasets, there has been much (informally) published past work. You should find, read and cite any past work in your prerecorded presentation and add these links to the supplemental materials you prepare, so that the instruction staff can check accordingly. You should validate (by replication) performance figures from other papers or posts. You do original work by going beyond what others have reported. There are many ways you can go beyond the past work in your analyses and subsequent iterative questioning and answering of your project work. Don't concentrate just on performance metrics but look for ways to connect what you've learned in lecture with your project.

   Examples include:

   - How do changes in your model architecture affect performance? Not just at the macro performance metrics but for individual (micro) and groups (meso) problem instances?
   - How does changing some input instances minimally change the results for better or worse?
   - Which features or model paradigm designs contribute the most towards performance and, more crucially, why?
   - Why do certain models do better at certain instances and not for others?

C Version History

Note: This list is in reverse chronological order. Timestamps follow the *yymmdd* format.

1. 250219 — Updated to explicitly cite AI tool use and to outdate information about reserved instances in the SoC cluster.
2. 240229 — Updated to as for audit trail regarding AI Tools Use (around Lines 480–485 in the PDF).
3. 230312 — Updated for Canvas and AI Tools policy.
4. 220324 — Updated report length and data analysis sections.
5. 220320 — Initial distribution.